

Programação *funcional*

Programação funcional é um estilo: montar o programa com **funções puras** (mesma entrada, mesma saída, sem mexer em nada de fora), **dados que não mudam** e funções que recebem e devolvem outras funções. O Python não é uma linguagem puramente funcional, mas oferece as ferramentas. Nesta aula: funções puras, evitar mutação, funções de alta ordem e composição, `reduce`, `partial`, `itemgetter` / `attrgetter` e um pipeline com `map`, `filter` e `reduce`.

Funções puras

Sem mutação

Alta ordem

Composição

`reduce` · `partial`

`itemgetter` · `attrgetter`

Formação Claude Code 2026 · Baseado em docs.python.org (Functional Programming HOWTO; functools: `reduce`, `partial`; operator: `itemgetter`, `attrgetter`; collections.namedtuple), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em [exemplos/](#) · Saídas reais

Mesma entrada, *mesma saída*

`puras.py`

```
total = 0

def somar_impura(valor):          # depende e altera algo de fora
    global total
    total += valor
    return total

def somar_pura(acumulado, valor): # só usa o que recebe
    return acumulado + valor

print(somar_impura(5), somar_impura(5)) # mesma entrada...
print(somar_pura(0, 5), somar_pura(0, 5)) # ...mesma saída
```

SAÍDA

```
5 10
5 5
```

- ✓ A impura muda a variável global `total`: chamada duas vezes com 5, responde 5 e depois 10
- ✓ A pura só usa os argumentos e não altera nada fora dela
- ✓ Funções puras são fáceis de testar, reaproveitar e entender isoladas

Todo programa precisa de efeitos (imprimir, gravar arquivo). A ideia é concentrá-los em poucos lugares e manter o resto puro.

Criar o novo, *sem alterar o antigo*

sem_mutar.py

```
notas = [7, 10, 5]

ordenadas = sorted(notas)      # cria uma lista nova
print(notas, ordenadas)       # a original não mudou

aluno = {"nome": "Ana", "nota": 7}
atualizado = {**aluno, "nota": 9}  # novo dicionário
print(aluno)
print(atualizado)

ponto = (2, 3)                 # tupla: não dá para alterar
print(ponto + (4,))            # cria outra tupla
```

SAÍDA

```
[7, 10, 5] [5, 7, 10]
{'nome': 'Ana', 'nota': 7}
{'nome': 'Ana', 'nota': 9}
(2, 3, 4)
```

ALTERA O ORIGINAL

CRIA UM NOVO

`lista.sort()`

`sorted(lista)`

`lista.append(x)`

`lista + [x]`

`d["chave"] = v`

`{**d, "chave": v}`

`list`, `dict`, `set`

`tuple`, `frozenset`, `str`

Dados que não mudam evitam o bug "quem alterou minha lista?" quando várias partes do código compartilham o mesmo objeto.

Funções que recebem *e devolvem funções*

alta_ordem.py

```
def aplicar_duas_vezes(funcao, valor):  
    return funcao(funcao(valor))  
  
def compor(f, g):  
    return lambda x: f(g(x))      # devolve uma função nova  
  
print(aplicar_duas_vezes(lambda n: n * 3, 2))  
  
limpar = compor(str.title, str.strip)  
print(repr(limpar("  maria da silva  ")))
```

SAÍDA

```
18  
'Maria Da Silva'
```

- ✓ `aplicar_duas_vezes` recebe uma função como argumento: $2 \times 3 \times 3 = 18$
- ✓ `compor(f, g)` devolve uma função nova que faz `f(g(x))`
- ✓ `map`, `filter`, `sorted(key=)` e os decoradores (aula 10) são funções de alta ordem

Compor funções pequenas é o coração do estilo funcional: cada peça faz uma coisa só.

Reduzir uma sequência *a um valor*

usando_reduce.py

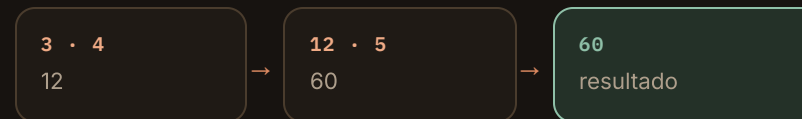
```
from functools import reduce
from operator import add, mul

nums = [3, 4, 5]

print(reduce(add, nums))           # ((3 + 4) + 5)
print(reduce(mul, nums))           # ((3 * 4) * 5)
print(reduce(lambda a, b: a if a > b else b, nums))
print(reduce(add, [], 0))          # valor inicial evita erro
```

SAÍDA

```
12
60
5
0
```



- ✓ `reduce(f, itens)` combina dois a dois, levando o acumulado adiante
- ✓ O terceiro argumento é o valor inicial: sem ele, lista vazia dá erro
- ✓ Para somar, contar ou achar o maior, prefira `sum`, `len` e `max`: são mais claros

Fixar argumentos: *funções sob medida*

usando_partial.py

```
from functools import partial

binario = partial(int, base=2)          # fixa o argumento base
print(binario("1011"), binario("11111111"))

def preco_final(valor, imposto):
    return round(valor * (1 + imposto), 2)

com_icms = partial(preco_final, imposto=0.18)
print(com_icms(100), com_icms(59.9))
```

SAÍDA

```
11 255
118.0 70.68
```

- ✓ `partial(função, ...)` devolve uma nova função com alguns argumentos já preenchidos
- ✓ `binario` é o `int` com `base=2` embutido
- ✓ Útil para passar funções "configuradas" a `map`, botões, callbacks

A alíquota de 18% é só um valor de exemplo.

itemgetter e attrgetter

getters.py

```
from operator import itemgetter, attrgetter
from collections import namedtuple

vendas = [("ana", 300), ("bia", 950), ("caio", 120)]
print(sorted(vendas, key=itemgetter(1), reverse=True))

Livro = namedtuple("Livro", "titulo ano")
livros = [Livro("Duna", 1965), Livro("1984", 1949)]
por_ano = sorted(livros, key=attrgetter("ano"))
print([livro.titulo for livro in por_ano])
```

SAÍDA

```
[('bia', 950), ('ana', 300), ('caio', 120)]
['1984', 'Duna']
```

- ✓ `itemgetter(1)` é uma função pronta que faz `x[1]`
- ✓ `attrgetter("ano")` faz `x.ano`
- ✓ Substituem lambdas como `lambda v: v[1]` no `key=`, com nome mais claro

map, filter e reduce

pipeline_funcional.py

```
from functools import reduce

pedidos = [120.0, 35.5, 980.0, 15.0, 410.0]

total_grandes = reduce(
    lambda acc, v: acc + v,
    map(lambda v: v * 0.9,          # 10% de desconto
        filter(lambda v: v >= 100, pedidos)),
    0,
)
print(round(total_grandes, 2))

# o mesmo, mais legível:
print(round(sum(v * 0.9 for v in pedidos if v >= 100), 2))
```

SAÍDA

```
1359.0
1359.0
```

FILTER

pedidos $\geq$ 100



MAP

10% de
desconto



REDUCE

soma

- ✓ As três etapas encaixadas dão o total dos pedidos grandes com desconto
- ✓ A versão com `sum` e expressão geradora dá o mesmo e é mais fácil de ler
- ✓ Em Python, use o estilo funcional quando ele deixar o código mais claro, não por regra

Compondo *várias funções*

Enunciado: escreva `compor(*funcoes)`, que aplica as funções em sequência, e use para normalizar um texto: tirar espaços das pontas, juntar espaços repetidos e passar para minúsculas.

Uma resposta possível

```
from functools import reduce
import re

def compor(*funcoes):
    """Aplica as funções da esquerda para a direita."""
    return lambda x: reduce(lambda acc, f: f(acc), funcoes, x)

def juntar_espacos(texto):
    return re.sub(r"\s+", " ", texto)

normalizar = compor(str.strip, juntar_espacos, str.lower)

print(repr(normalizar(" Olá    MUNDO Python  ")))
```

SAÍDA

```
'olá mundo python'
```

- ✓ `reduce` passa o texto por cada função, da esquerda para a direita
- ✓ Cada etapa é uma função pura e pequena, fácil de trocar
- ✓ Desafio: acrescente uma etapa que remove acentos

Teste o que *ficou*

1. Uma função pura:

A) Altera variáveis globais

B) Para a mesma entrada, dá sempre a mesma saída

C) Não pode ter return

D) Precisa ser lambda

B.

2. Qual NÃO altera a lista original?

A) lista.sort()

B) lista.append(1)

C) sorted(lista)

D) lista.pop()

C.

3. `reduce(lambda a, b: a + b, [1, 2, 3])` vale:

A) [1, 3, 6]

B) 6

C) 3

D) Erro

B.

4. `partial(int, base=16)("ff")` devolve:

A) "ff"

B) 255

C) 16

D) Erro

B.

Funções pequenas, *dados que não mudam*

Prefira funções puras e crie dados novos em vez de alterar os antigos. Funções são valores: passe, devolva e componha. `reduce` resume uma sequência, `partial` fixa argumentos e `itemgetter` / `attrgetter` substituem lambdas simples. Use o estilo funcional onde ele deixa o código mais claro.

Pura

Imutável

Alta ordem

`reduce` · `partial`

`operator`

Próxima aula → 14 · Usando Type Hint

Fontes: docs.python.org/3/howto/functional.html; docs.python.org/3/library/functools.html (`reduce`, `partial`); docs.python.org/3/library/operator.html (`add`, `mul`, `itemgetter`, `attrgetter`); docs.python.org/3/library/collections.html#collections.namedtuple, consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.